Streaming RL with Memory for POMDPs

Malinin Andrey [1]

Abstract

We study reinforcement learning on a partially observable gridworld benchmark, MiniGrid-Memory S7. We implement four agents: a streaming actor-critic (Stream-x), its recurrent variant (Stream-x+LSTM), batch PPO, and PPO+LSTM. Across runs, all implementations converge to a similar mean episodic reward of approximately 0.5. For memoryless agents this level is consistent with a theoretical limitation in the POMDP caused by observation aliasing. All implementation were firstly launched for simpler memory test environment, where all "memory" solutions showed mean 1.0 reward. However, for MiniGrid-Memory S7 recurrent agents do not surpass this plateau, suggesting that the learned memory is not effectively used; we discuss likely causes and practical checks.

1. Introduction

Partially observable environments require agents to aggregate information across time. A common approach is to add trainable memory (e.g., an LSTM), which can in principle transform a POMDP into an effectively fully observable problem if the hidden state is inferable from history. In this report we focus on a single assignment environment, **MiniGrid-Memory S7**, and compare four implemented baselines: Stream-x / Stream-x + LSTM and PPO / PPO + LSTM.

The main empirical observation is that **all four agents plateau around** 0.5 **mean episodic reward**. This is expected for reactive policies in this benchmark, but unexpected for recurrent agents, which ideally should break the reactive limit.

[1]Moscow Institute of Physics and Technology. Correspondence to: Malinin Andrey <malinin.aa@phystech.edu>.

2. MiniGrid-Memory S7 and the Reactive Performance Limit

MiniGrid-Memory S7 is a small POMDP where a cue observed earlier in the episode is needed to choose correctly later. Formally, the environment has latent state s_t, observation o_t, action a_t, and reward r_t. A memoryless policy chooses actions as $a_t \sim \pi(a_t \mid o_t)$, while a recurrent policy maintains an internal state

$$h_t = f_\theta(h_{t-1}, o_t), \qquad a_t \sim \pi_\theta(a_t \mid h_t). \tag{1}$$

Why ≈ 0.5 **is reasonable without memory.** The key limitation is **observation aliasing**: two different latent situations can produce the same observation but require different actions. Consider a critical decision point where: (i) the agent receives the *same* observation under two latent cases, and (ii) the correct action differs between the cases. Any reactive policy must behave identically in both cases, so it cannot always pick the correct action. If the two cases occur with roughly equal probability, the best possible success probability at that decision point is about 0.5, yielding an episodic reward ceiling near 0.5 when the episode reward is driven by that decision. Thus, the observed plateau for stream-x and PPO *without* memory matches the expected reactive optimum for this POMDP.

3. Implemented Algorithms

We implemented four agents for the same task:

- **Stream-x**: streaming actor-critic trained online (no replay buffer, no minibatches).
- **Stream-x + LSTM**: stream-x with recurrent policy/value networks.
- **Batch PPO**: standard PPO with feed-forward networks.
- **Batch PPO + LSTM**: PPO trained on recurrent sequences (BPTT).

4. Experimental Setup

We report **mean episodic reward** as a function of environment steps (or episodes, depending on the implementation).

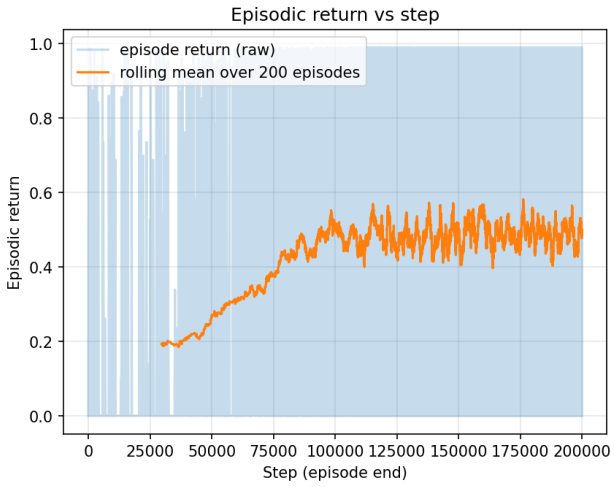

Figure 1. Learning curve on MiniGrid-Memory S7 for stream-x (memoryless).

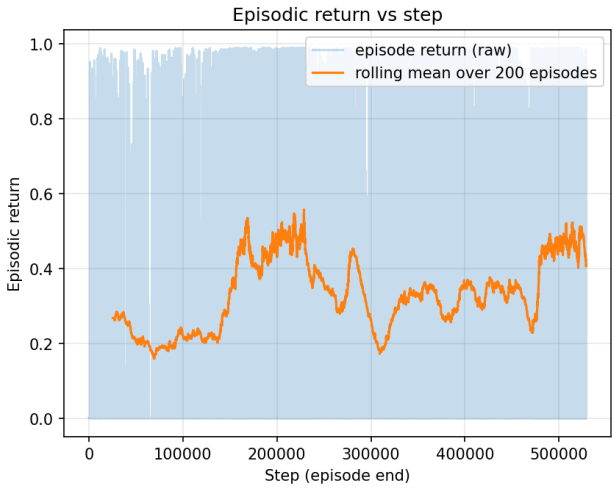

Figure 2. Learning curve on MiniGrid-Memory S7 for stream-x + LSTM.

The four learning curves are provided as separate plots.

5. Results

Figures 1–4 show reward vs training progress for each implementation. All methods converge near a mean episodic reward of ≈ 0.5.

Interpreting the plateau. For the memoryless agents (stream-x, PPO), the ≈ 0.5 reward is consistent with the reactive upper bound due to POMDP. In contrast, the recurrent agents (stream-x+LSTM, PPO+LSTM) *also* plateau at ≈ 0.5, implying that the learned recurrence did not consistently capture and exploit the early cue needed for near-perfect decisions later.

6. Why did LSTM not exceed the reactive limit?

Below we provide explanations that are consistent with the observed behavior.

6.1. Insufficient effective temporal credit assignment

Even if the cue is observed early, the learning signal for "remembering" it must propagate back through time to the parameters that update h_t. If training uses truncated back-propagation through time (TBPTT) with a window shorter than the cue-to-decision delay, gradients reaching the relevant time steps become weak or biased. In this case, the LSTM can behave almost reactively, reproducing the ≈ 0.5 ceiling.

6.2. Recurrent optimization sensitivity and hyperparameter mismatch

Recurrent RL is typically more sensitive to learning rates, gradient clipping, and value loss scaling. In particular, the stream-x+LSTM implementation uses a relatively large head learning rate ($lr\_head = 1.0$), which can make optimization noisy and destabilize the delicate memory dynamics. With high variance gradients, the optimizer may converge to an "easy" reactive strategy that yields stable ≈ 0.5 reward, rather than learning a harder memory-dependent solution.

6.3. Exploration and data coverage

To learn memory, the agent must experience trajectories where different cues lead to different optimal actions, and must receive enough training signal on both cases. If exploration is limited or early training collapses to a single mode of behavior, the data may not contain sufficient informative transitions for the LSTM to discover a useful internal representation. The agent can then settle at the robust reactive optimum.

6.4. Implementation pitfalls that can mask memory benefits

Finally, several practical issues can prevent an LSTM from helping even if it is present:

- Training is unstable and highly dependant on hyperparameters making learning process complicated and less predictable.
- Frequent detaching of the hidden state (or too small

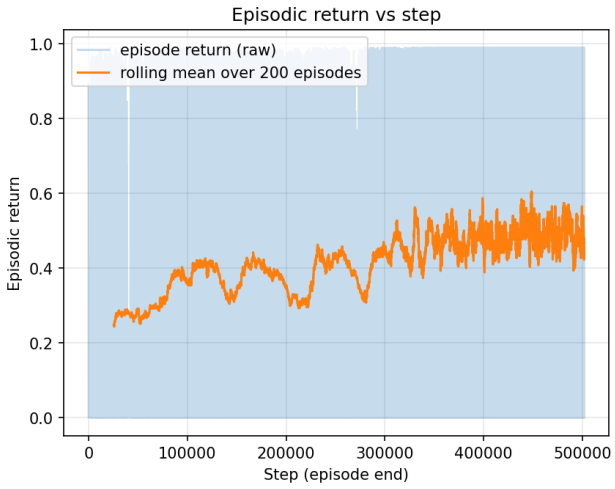

Figure 3. Learning curve on MiniGrid-Memory S7 for batch PPO (memoryless).

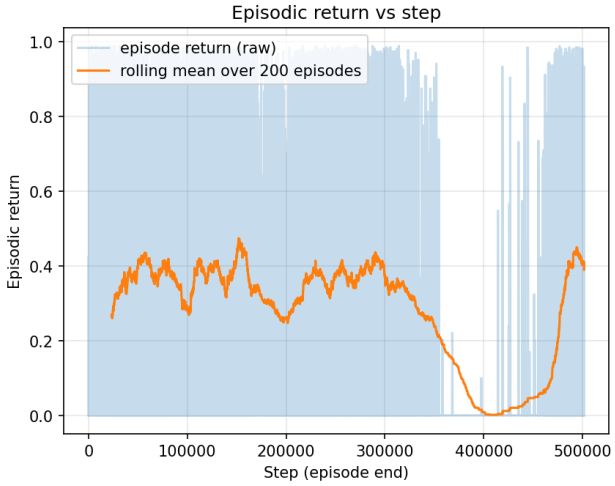

Figure 4. Learning curve on MiniGrid-Memory S7 for batch PPO+LSTM.

TBPTT window) removes long-horizon gradients.

- Actor is recurrent but critic is not (or vice versa), increasing advantage noise and making memory learning harder.

7. Conclusion

We implemented stream-x and PPO baselines on MiniGrid-Memory S7, with and without LSTM memory. All methods converged to ≈ 0.5 mean episodic reward. For memoryless agents this matches the expected reactive limitation of the POMDP due to observation aliasing. Recurrent agents did not surpass this plateau, likely due to long-horizon credit assignment difficulties, recurrent optimization sensitivity and/or limited exploration. A practical next step is to validate recurrent state handling, increase effective unroll horizons (TBPTT/sequence length), and tune recurrent-specific stabilization (learning rates, clipping, and value/entropy coefficients).

Table 1. Main hyperparameters for MiniGrid-Memory S7.

	Stream-x	Stream-x+LSTM	PPO	PPO+LSTM
γ	0.99	0.99	0.99	0.99
λ	0.8	0.8	0.95	0.95
LR (main)	1.0	1.0	2.5×10^{-4}	2.5×10^{-4}
Entropy	0.01	0.01	0.01	0.01
Clip ϵ	–	–	0.2	0.2
Grad norm	–	1.0	0.5	0.5
BPTT / seq	–	80	–	32